SIMATS ENGINEERING

Department of Information Technology

CSA37 – Software Testing

ASSIGNMENT 1 – CO1

Evaluate the Quality Process and Defect Profile of a Hospital Appointment Application

CO1: Testing viewed as an engineering activity, the role of process in software quality, testing as a process, basic definitions, principles of software testing, the role of the tester, origins of defects, defect classes, defect repository and defect examples.

Student Name	P.Ranjithkumar	**Register Number**	192411119
Department / Section	BE.CSE	**Academic Year / Semester**	2024
Faculty Name	Dr.Kesavan R	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	1.11.2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Evaluate Testing as an Engineering Activity	5	**5**	All four engineering features (planning, measurement, standards, repeatability) judged against case evidence and mapped to incidents; engineered approach clearly stated.
2	Evaluate the Role of Process in Software Quality	2	**2**	Clear process-over-people position; Table 2 tests each incident; 38/100 coding-defect argument and TMM Level 1 placement well justified.
3	Evaluate Testing as a Process	6	**6**	Full test process, V&V per phase, detection phase per incident; cost ₹26,10,500 computed correctly; defect-flow model and limits explicitly stated.
4	Apply and Critique Basic Definitions	4	**4**	Definitions applied to MediQueue; error/fault/failure for I3, I5, I6 correct; I8 linked to a wrong oracle; glossary contents justified.
5	Evaluate the Application of Testing Principles	3	**3**	Six principles each tied to an incident or decision; “testing shows presence, not absence” used well; strongest principle defended with cost data.
6	Evaluate the Tester’s Role	3	**3**	Clear position; tester role per phase, pros/cons of self-testing, 7/2/1 catch table and healthcare-specific tester qualities.
7	Trace the Origins of Defects	4	**4**	Origin and evidence for all ten incidents with second-origin reasoning; pattern (50% requirements/design) matches the defect data.

8	Evaluate Defect Classes and Severity	4	**4**	Classification scheme with sub-classes; class, severity and priority for all ten; I7 vs I1 contrast explains severity ≠ priority.
9	Design and Appraise a Defect Repository	3	**3**	Field groups justified individually; analysis/improvement/prevention uses and e-mail vs repository comparison are convincing.
10	Evaluate Defect Examples and Prevention Strategies	4	**3.5**	Three classes, root causes and prevention strategies correct. Cost model rests on assumed ₹32,000 escape cost; overlapping savings noted but not quantified.
11	Recommend and Defend a Quality Improvement Plan	4	**3.5**	Prioritised, practical plan with 3.2% effort check. Expected effect depends on assumed 60%/35%/60% efficiencies that are not validated.
12A	Re-evaluate: Impact Analysis	2	**2**	Change table and affected assumptions clear; effect of two-week releases and shared records on severity well analysed.
12B	Re-evaluate: Evaluation of the Original Plan	2	**2**	Capacity check (184 vs 140 hours, 31% over) supports the conclusion; new external-system defect origins and repository gaps identified.
12C	Re-evaluate: Adaptation or Redesign	2	**2**	Clear MODIFY decision; revised two-week cycle, tester role, principles and external-defect classification (Table 26) shown.
12D	Re-evaluate: Evidence-Based Defence	2	**2**	Decision defended with defect, cost, capacity and patient-safety evidence; accepted risks and review trigger stated.
	TOTAL	**50**	**49**	**Percentage: 98%**

OVERALL REMARKS

Excellent, well-evidenced submission. Every task is answered with case data, the cost and defect-flow calculations are correct and internally consistent (₹26,10,500; 60 production escapes; 9.6% early detection), and the Day-4 redesign follows logically from a capacity calculation. Minor weaknesses: the savings in Tasks 10–11 depend on assumed review/test efficiencies and cost figures that are stated but not validated, and the solution repeats each task's bullet list, which adds length without adding marks.

Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A well-argued assessment of MediQueue's testing practice against the features of an engineering activity, backed by evidence from the case.	✓	Tables 1–2 and incident mapping
2	A well-justified position on how process affects quality, backed by data from the case.	✓	TMM placement justified
3	A process-oriented assessment of testing in MediQueue, backed by cost calculations from the defect data and a reasoned estimate of savings.	✓	Calculations shown; Figure 1 present
4	Accurate application of basic testing definitions to the MediQueue incidents and a justified judgement on how terminology affects quality.	✓	I3, I5, I6 and I8 covered
5	A justified assessment of the developers' assertion using testing principles tied to evidence from the case.	✓	Six principles linked to evidence
6	A defended judgement on how valuable an independent tester would be for MediQueue, tied to specific incidents.	✓	Position defended with catch table
7	A justified origin for every incident and an interpretation of the overall pattern of defect origins.	✓	All ten incidents traced
8	A justified classification of all ten incidents by class, severity and priority, together with a recommendation on which class to address first.	✓	Severity vs priority contrast given
9	A defect repository design for MediQueue with a justification for every field and an assessment of its benefits.	✓	Each field group justified
10	A root-cause analysis of three defects with justified prevention strategies that take cost into account.	**P**	Savings rest on assumed costs
11	A technically justified quality improvement plan backed by the evidence built up in earlier tasks.	**P**	Efficiencies assumed, not validated
12A–D	Impact analysis, evaluation of the original solution, adaptation or redesign, and evidence-based defence	✓	All four parts complete

DEFECT ANALYSIS EVIDENCE CHAIN (Tasks 3, 7–10)

Evidence item	Status	Observation
Defect data calculations (cost per phase, totals)	✓	Cost per phase and totals correct
Phase-containment / detection figures	✓	Table 7 flow; 40% detection before release

Origin assigned to every incident (I1–I10)	✓	I1–I10 with evidence and second origin
Class, severity and priority justified	✓	Table 17 with reasons
Defect repository fields with reasons	✓	Table 18 with reasons
Root cause → prevention strategy link	✓	Table 20
Cost–benefit of prevention strategies	**P**	Ratios 12.6 and 8.8 depend on assumptions
Interpretation of the defect pattern	✓	Pareto and investment conclusion
Figures and conclusions consistent	✓	Figures consistent across Tasks 1–12

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☒ Yes ☐ Partly ☐ No
Defect cost calculations shown step by step	☒ Yes ☐ Partly ☐ No
All ten incidents classified (origin, class, severity, priority)	☒ Yes ☐ Partly ☐ No
Error, fault and failure separated for I3, I5 and I6	☒ Yes ☐ Partly ☐ No
Testing principles linked to evidence from the case	☒ Yes ☐ Partly ☐ No
Repository design justified field by field	☒ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☒ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☒ Yes ☐ Partly ☐ No
Work consistent throughout	☒ Yes ☐ Partly ☐ No
No unfilled template placeholders	☒ Yes ☐ Partly ☐ No
Viva recommended	• Yes ☒ No

Software Testing

ASSIGNMENT 2

CO2: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (Total Marks - 50)

Duration:1week

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	**Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.**
S2	**Charge is calculated for every started kilogram: Zone A ₹40/kg, Zone B ₹60/kg, Zone C ₹90/kg. Any other zone is INVALID.**
S3	**Express delivery adds 50 % to the charge. Express is allowed only up to 30 kg; above 30 kg the request is NOT_ALLOWED.**

S4	**Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.**

Shipping-charge module – Pseudocode under test:

```
function shipCharge(weight, zone, express, parcels)
 1  if weight <= 0 or weight > 50 then
 2    return INVALID
 3  rate = 40
 4  if zone == "B" then
 5    rate = 60
 6  else if zone == "C" then
 7    rate = 90
 ●  charge = 0
 ●  w = weight
●   while w > 0 do
11      charge = charge + rate
12    w = w - 1
13  if express then
14    if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Parcel tracking states:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED.
A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge:

SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified.

Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

TASK 1 – Select Test Case Design Strategies 5 Marks

CO2 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module.

Which test design strategy will find the most important defects in each module with the least effort?

Your answer must include:

- **The difference between black-box and white-box test design strategies**
- **The strategy you select for the shipping-charge rules, for the pseudocode and for the tracking states**
- **The information each strategy needs (specification, code, state model)**
- **The kinds of defects each strategy is likely and unlikely to reveal**
- **A justification for combining strategies rather than using only one**

You are expected to select and justify strategies independently for this system.

Evidence Expected

A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

TASK 2 – Evaluate Random Testing 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases.

Can random testing replace systematic test design for shipCharge? Discuss:

- **How random testing would be carried out for shipCharge**
- **The probability that random inputs hit important values such as 30.0, 30.1 and 50.0**
- **The problem of deciding the expected result (test oracle) for random inputs**

Your judgement must be supported by reasoning, not opinion. Evidence

Expected

A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

TASK 3 – Design and Evaluate Equivalence Classes 6 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning.

What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Discuss:

- **Valid and invalid equivalence classes for weight, zone, express and parcels**
- **How you identified each class from the specification**
- **Test cases covering all valid classes with as few tests as possible**
- **Test cases covering each invalid class one at a time, and why**
- **Expected results calculated from the specification**
- **Which of your test cases would fail against the pseudocode, and what that reveals**

Simply listing classes will not receive full credit. You are expected to evaluate the effectiveness of your classes.

Evidence Expected

A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

TASK 4 – Design and Evaluate Boundary Value Tests 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges.

Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Discuss:

- **Boundary values for weight (0, 0.1, 30, 30.1, 50, 50.1), the** started-kilogram rule and parcels (9, 10)
- **Normal BVA, robust BVA and worst-case BVA, and the number of test cases each produces**
- **Test cases with expected results for the boundaries you select**
- **Which form of BVA you recommend for shipCharge, and why**

You must justify the choice of BVA form against the number of test cases and the risk.

Evidence Expected

A justified boundary value test design with expected results and a defended choice of BVA form.

TASK 5 – Apply Other Black-Box Techniques 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other.

Do ECP and BVA miss defects caused by combinations of rules? Discuss:

- **A cause–effect graph or decision table for rules S3 and S4**
- **Test cases derived from the decision table**
- **Combinations that ECP/BVA did not cover, and whether they matter**

You are expected to compare the results with your ECP/BVA test set.

Evidence Expected

A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

TASK 6 – Design and Evaluate State-Transition Tests 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel.

Which invalid transitions could cost the company money or reputation if they were allowed?

Discuss:

- **A state-transition diagram and table for parcel tracking**
- **Test cases for all valid transitions and selected invalid transitions**
- **Tests for the "maximum two retries" rule and the cancellation rule**

You are expected to justify which invalid transitions you selected.

Evidence Expected

A state-transition model with justified valid and invalid transition tests.

TASK 7 – Construct the Control Flow Graph 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph. Does the

structure of the code match the structure of the specification? Discuss:

- **Nodes and edges, with line numbers mapped to nodes**
- **Decision nodes, the loop, and the compound condition at line 1**
- **Differences between the code structure and the rules S1–S4 (for example, invalid zones)**

The graph must be derived from the code, with the mapping clearly shown. Evidence Expected

A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code.

How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Discuss:

- **Cyclomatic complexity calculated by at least two methods**
- **A set of independent (basis) paths and test data for each path**
- **Infeasible paths, if any, and why**
- **How the loop affects path testing, and how you would test it (zero, one, many iterations)**

You must show the derivation. Stating only the final number will not receive full credit.

Evidence Expected

A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

TASK 9 – Judge Test Adequacy and Coverage 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is "enough".

Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Discuss:

- **Statement, branch, condition and multiple-condition coverage for shipCharge**
- **The coverage achieved by your path tests from Task 8**
- **The additional tests needed to reach the next stronger criterion**
- **Whether 100 % coverage would have found the invalid-zone defect**

Your recommended criterion must be defended against cost and risk.

Evidence Expected

A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

TASK 10 – Compare Black-Box and White-Box Effectiveness 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9).

Which approach actually found more of the important defects in shipCharge? Discuss:

- **The defects found by black-box tests only, white-box tests only, and both**
- **Reasons why each approach missed what it missed**
- **Overlapping tests that can be removed without losing defect-finding power**
- **Whether this result would hold for other modules**

You are expected to interpret the comparison, not just count tests. Evidence

Expected

A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

TASK 11 – Recommend and Defend the Final Test Suite 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Discuss:

- **The test cases you include and the technique each comes from**
- **Why this suite is better than the alternatives**
- **The coverage and the rule combinations it achieves**
- **The number of test cases and the effort needed to run them**
- **Known gaps and risks that remain**

Your recommendation must be based on evidence from your own test design. The largest suite is not automatically the best.

Evidence Expected

A technically justified final test suite supported by black-box, white-box and coverage evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Test Suite 8 Marks

CO2 | Bloom's Level: K5 – Evaluate

Changed Requirement

SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds ₹50 per parcel.

The developers have changed shipCharge by adding two new decisions after line 16.

The suite must still run within 30 minutes before every release.

Re-evaluate your original solution in response to these changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed in the specification and the code**
- **Which equivalence classes, boundaries and decision-table rules are affected**
- **How the control flow graph and cyclomatic complexity change**
- **Why the 30-minute limit matters**

Evidence Expected

A clear analysis of how the changes affect the original test design.

B. Evaluation of the Original Suite – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Evaluate whether your final test suite remains adequate. Discuss:

- **Which existing tests are still valid, need new expected results, or are obsolete**
- **Which new paths and rule combinations are not covered**
- **Whether the coverage criterion can still be met**

Evidence Expected

A technically supported evaluation of the original suite under the new conditions.

C. Adaptation or Redesign – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the test suite. If you

modify or redesign it:

- **Describe the new and changed test cases**
- **Explain which techniques you used to derive them**
- **Show the effect on coverage and the number of test cases**

If you retain the original suite, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate test-design response to the changed requirement with clear reasoning.

D. Evidence-Based Defence – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defend your final test suite using evidence. Use:

- **Equivalence classes and boundaries**
- **Decision-table combinations**
- **Cyclomatic complexity and basis paths**
- **Coverage achieved**
- **Execution time limit**

Your final decision must follow logically from your analysis. Evidence Expected

A well-supported final test suite integrating test design evidence and the changed constraints

---

MARK DISTRIBUTION

Task	Marks
Task 1 – Select Test Case Design Strategies	**5**
Task 2 – Evaluate Random Testing	**2**
Task 3 – Design and Evaluate Equivalence Classes	**6**
Task 4 – Design and Evaluate Boundary Value Tests	**4**
Task 5 – Apply Other Black-Box Techniques	**3**
Task 6 – Design and Evaluate State-Transition Tests	**3**
Task 7 – Construct the Control Flow Graph	**4**
Task 8 – Evaluate Path Testing and Cyclomatic Complexity	**4**
Task 9 – Judge Test Adequacy and Coverage	**3**
Task 10 – Compare Black-Box and White-Box Effectiveness	**4**
Task 11 – Recommend and Defend the Final Test Suite	**4**
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

Software Testing (CSA3711)

ASSIGNMENT 2

CO2 Coverage: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Student Name	Register Number	Department / Section
P.Ranjithkumar	192411119	BE.CSE

Academic Year / Semester	Faculty Name	Date of Submission
2024	Dr.Kesavan R	1.11.2026

Nature of Assignment: Individual **Duration:** 1 Week **Total Marks:** 50

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (Total Marks - 50)

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.
S2	Charge is calculated for every started kilogram: Zone A ₹40/kg, Zone B ₹60/kg, Zone C ₹90/kg. Any other zone is INVALID.
S3	Express delivery adds 50 % to the charge. Express is allowed only up to 30 kg; above 30 kg the request is NOT_ALLOWED.
S4	Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.

Shipping-charge module – pseudocode under test:

```
function shipCharge(weight, zone, express, parcels)
 1  if weight <= 0 or weight > 50 then
 2      return INVALID
 3  rate = 40
 4  if zone == "B" then
 5      rate = 60
 6  else if zone == "C" then
 7      rate = 90
 8  charge = 0
 9  w = weight
10  while w > 0 do
11      charge = charge + rate
12      w = w - 1
13  if express then
14      if weight > 30 then
```

```
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Parcel tracking states: BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge: SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified. Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

Task 1 – Select Test Case Design Strategies **5 Marks**

CO2 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module. Which test design strategy will find the most important defects in each module with the least effort?

Evidence Expected: A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

Ans:

Choosing a Strategy for Each Part of SwiftShip

I choose by asking three questions for every part: what information do I have, what kind of defect would cost the company most, and which technique reaches that defect with the fewest tests? The answers differ for the charge rules, the code, and the tracking states, so I use a different strategy for each and justify the combination at the end.

Black-box and white-box design

Table 1: Black-box compared with white-box test design

Aspect	Black-box (specification-based)	White-box (code-based)
Basis	Test cases come from the specification (S1–S4, state model); the code is ignored	Test cases come from the code structure (control flow, conditions, loops)
Question asked	"Does the program do what the rules say?"	"Has every part of the code been exercised?"
Needs	Specification only; can start before code exists	Code or pseudocode; can start only after design/coding
Strong at	Missing functions, wrong rules, boundary errors, rule combinations	Unexecuted statements, untested branches and loops, dead code
Weak at	Code that does extra things nobody specified; hidden paths	Code that is missing (omissions); wrong rules that the code implements faithfully
Adequacy measure	Coverage of classes, boundaries, rules, transitions	Statement, branch, condition and path coverage

Strategy selected for each part

Table 2: Strategy selected, information required and expected defects

Part under test	Strategy selected	Information needed	Defects likely to be revealed	Defects unlikely to be revealed

Part under test	Strategy selected	Information needed	Defects likely to be revealed	Defects unlikely to be revealed
Charge rules S1–S4 (what the module must do)	Black-box: equivalence class partitioning, boundary value analysis, decision table	Specification S1–S4 only	Missing validation (invalid zone), wrong limits (30, 50), wrong rate, rule combinations such as express with discount	Faults in code paths the specification does not mention; floating-point effects
shipCharge code (how it does it)	White-box: control flow graph, basis-path testing, loop testing, branch and condition coverage	Pseudocode (lines 1–19)	Untested branches, loop errors (zero, one, many iterations), wrong conditions such as > for >=	Omitted logic (no code to execute), wrong rules coded faithfully
Parcel tracking states	Black-box, model-based: state-transition testing	State model and transition rules (booking → delivery, retries, cancellation)	Illegal transitions allowed, retry limit not enforced, cancellation after pick-up	Calculation errors; defects not tied to a state change

Why the strategies are combined

No single strategy covers the whole system. The sharpest example is already visible in the pseudocode: the specification says any zone other than A, B or C is INVALID (S2), but the code gives such a zone the Zone A rate. A white-box test set that executes every statement and every branch passes straight over this defect, because the "invalid zone" code does not exist; only a specification-based test with zone "D" exposes it. In the other direction, black-box tests choose data from the specification and cannot tell whether the loop at lines 10–12 was exercised for 1, 2 and 50 iterations, or whether the compound condition at line 1 was ever evaluated with weight > 50 as the reason. Combining the two gives a check on each side, and Task 10 measures this with seeded defects (black-box found 15 of 15, white-box 11 of 15, and the two together found all).

The tracking module is a separate case: it has a state model but no calculation, so state-transition testing gives the best return, and neither ECP nor path testing would find an illegal transition such as CANCELLED → PICKED_UP.

Effort. The selection keeps the suite small: black-box design yields a few dozen cases for the rules, white-box analysis adds only the cases that cover loops and conditions left over, and the state model adds about a dozen. Random testing (Task 2) is kept only as a supplement.

Task 2 – Evaluate Random Testing **2 Marks**

CO2 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases. Can random testing replace systematic test design for shipCharge?

Evidence Expected: A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

Ans:

Can Random Testing Replace Systematic Design?

My position: **no**. Random testing is cheap and has a place as a supplement, but it cannot replace designed tests for shipCharge because it rarely lands on the values where defects hide, and it needs a trustworthy oracle for every one of its 1,000 results.

How random testing would be carried out

- A generator draws weight uniformly from the allowed one-decimal values 0.1 … 50.0 (500 values), zone from {A, B, C} (and possibly other letters), express true/false and parcels from 0 to 30.
- The program is run 1,000 times and each result is compared with an expected value computed independently from S1–S4 (for example, a reference calculator written from the specification).
- To test invalid input the generator must also be widened, for example to −10.0 … 60.0 and any letter, which lowers the chance of each valid value.

Probability of hitting the important values

With 500 equally likely weights, the chance that one particular draw equals a given value is 1/500. The chance that 1,000 draws contain it at least once is 1 − (499/500)^1000.

Table 3: Chance that 1,000 random tests hit the important values

Target	Assumed generator	Chance per draw	Chance in 1,000 draws
Weight = 30.0 (or 30.1, or 50.0), one value	500 valid weights	1/500 = 0.20%	86.5%
All three of 30.0, 30.1 and 50.0	500 valid weights	—	64.7%
Weight = 0.0 (lower invalid edge)	701 weights from −10.0 to 60.0	1/701 = 0.14%	76.0%
Weight = 30.1 and express = true	500 weights × 2 express values	1/1,000 = 0.10%	63.2%
Weight = 30.1, express = true and parcels = 10	500 × 2 × 31 parcels (0–30)	1/31,000 = 0.003%	3.2%

Even for single boundary values the chance of missing at least one of the three is about 35%, and for realistic combinations of boundary value, express flag and discount threshold it is very high. A designed suite hits each of these on purpose in 3 to 6 tests. Random draws also waste effort: with 1,000 uniform weights, about 600 will fall in the middle of the range (10–40 kg) where the code behaves identically, giving the same coverage as 5 or 6 designed tests.

The oracle problem

- Every random input needs an expected result. Without a hand-calculated value the tester must write a second implementation of the rules; if it is copied from the code, it repeats the code's defects (for example, treating zone "D" as Zone A), and if it is written independently it is itself a piece of software that needs testing.
- The result is a decimal: 1.5 and 0.9 multiplications may leave floating-point differences, so the comparison needs a tolerance, and the specification does not say how to round to paise.
- 1,000 results cannot be inspected by hand, so failures depend wholly on that automated oracle.

Judgement. Random testing is useful as a cheap supplement: once a reference calculator exists, it can run thousands of overnight comparisons to catch crashes and unexpected differences. But it cannot replace systematic design, because it gives no guarantee of boundary, rule-combination or invalid-class coverage, and its value depends entirely on an oracle that systematic design (Tasks 3–5) must produce anyway.

Task 3 – Design and Evaluate Equivalence Classes 6 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning. What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Evidence Expected: A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

Ans:

Equivalence Classes for the Shipping-Charge Rules

I derive classes from the specification alone. For each input I look for the points at which the specification changes the behaviour; values on the same side of each point are expected to be treated alike. S3 matters for weight even though it concerns express, because at 30 kg the allowed behaviour changes, so weight has two valid classes, not one.

Table 4: Equivalence classes derived from S1–S4

ID	Input	Class	Valid / invalid	Derived from
W1	Weight	0 < weight ≤ 30 (express allowed)	Valid	S1 range, S3 limit of 30 kg
W2	Weight	30 < weight ≤ 50 (express not allowed)	Valid	S1 range, S3 limit of 30 kg
W3	Weight	weight ≤ 0	Invalid	S1 "greater than 0"
W4	Weight	weight > 50	Invalid	S1 "not more than 50"
K1	Weight basis	Whole kilogram (for example, 10.0)	Valid	S2 "started kilogram"
K2	Weight basis	Part kilogram (for example, 2.5)	Valid	S2 "started kilogram"
Z1 / Z2 / Z3	Zone	A / B / C	Valid	S2 three rates
Z4	Zone	Any other letter (for example, "D")	Invalid	S2 "any other zone is INVALID"
Z5	Zone	Blank or missing zone	Invalid	S2 "any other zone"
E1 / E2	Express	True / false	Valid	S3
E3	Express with weight	Express true and weight > 30	Invalid (NOT_ALLOWED)	S3 limit
P1	Parcels this month	0 to 9 (no discount)	Valid	S4
P2	Parcels this month	10 or more (10 % discount)	Valid	S4
P3	Parcels this month	Negative number	Invalid (specification silent)	Implied by S4; expected result must be confirmed

That makes 17 classes in all (counting Z1, Z2, Z3 and E1, E2 separately): 11 valid and 6 invalid (W3, W4, Z4, Z5, E3, P3).

Valid classes covered with the fewest tests

Valid classes of different inputs can share one test, so the smallest number of valid tests equals the largest number of valid classes in any single input. Zone has three (A, B, C), so **three tests** are the minimum. I choose values so that every other valid class is also covered.

Table 5: Test cases from the equivalence classes, with expected results (specification) and results against the pseudocode

Test	Weight	Zone	Express	Parcels	Classes covered	Expected result (calculation)	Result on code
TC1	2.5	A	No	5	W1, K2, Z1, E2, P1	3 kg × 40 = 120	120 Pass
TC2	10.0	B	Yes	12	W1, K1, Z2, E1, P2	10 × 60 = 600; × 1.5 = 900; × 0.9 = 810	810 Pass
TC3	40.0	C	No	0	W2, K1, Z3, E2, P1	40 × 90 = 3,600	3,600 Pass
TC4	−2.0	A	No	0	W3	INVALID	INVALID Pass
TC5	60.0	A	No	0	W4	INVALID	INVALID Pass
TC6	5.0	D	No	0	Z4	INVALID	**200 FAIL**
TC7	5.0	(blank)	No	0	Z5	INVALID	**200 FAIL**
TC8	35.0	A	Yes	0	E3	NOT_ALLOWED	NOT_ALLOWED Pass
TC9	5.0	A	No	−1	P3	Not defined; reject expected	Returns 200: gap to be confirmed

Invalid classes are tested one at a time: in TC4 to TC9 every other input is valid, so a failure can be traced to one cause and one invalid value cannot hide behind another. If TC6 had also used weight −2.0, the INVALID result from the weight would have concealed the zone defect. That gives **three valid and six invalid tests, nine in all**, for 17 classes.

Which tests fail against the pseudocode, and what that reveals

TC6 and TC7 fail. The code (lines 3–7) sets rate = 40 and changes it only when the zone is "B" or "C", so any other zone, including blank, is charged as Zone A (₹200 for 5 kg) instead of being rejected. The fault is an **omission**: the INVALID return required by S2 does not exist. The patient impact is direct: a mistyped zone is accepted at the cheapest rate, causing undercharges on part of 40,000 daily parcels.

TC9 shows a different kind of finding: the specification does not say what happens with a negative parcel count, and the code simply gives no discount. This is a specification gap, which I raise as a query rather than a code defect.

Evaluation of the classes

- **Effectiveness:** 9 tests represent 17 classes and expose the one real defect in the code (2 of the 9 tests fail, and both point to the same fault).
- **What passed is also evidence:** TC1–TC5 and TC8 show that weight validation, the started-kilogram calculation, express and discount behave correctly for typical values in each class.
- **Limit:** ECP assumes every value in a class behaves alike, so it does not find edge errors (a ">" written as ">="); that is why Task 4 adds boundary values. Task 10 confirms this: this ECP set missed all five boundary faults that I seeded.

Task 4 – Design and Evaluate Boundary Value Tests **4 Marks**

CO2 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges. Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Evidence Expected: A justified boundary value test design with expected results and a defended choice of BVA form.

Ans:

Boundary Value Analysis for shipCharge

Boundaries are where one wrong comparison operator changes the charge for many parcels. I list every point at which the specification or the code changes behaviour, then rank them by risk.

Table 6: Boundaries in SwiftShip and why each is risky

Boundary	Values to test	Why it can hide a defect	Risk
Weight lower limit (S1)	0.0 (invalid), 0.1 (smallest valid)	Line 1 uses <= 0; "< 0" would accept zero weight and charge 0 or wrong amount	High
Weight upper limit (S1)	50.0 (valid), 50.1 (invalid)	Line 1 uses > 50; ">= 50" would reject the heaviest valid parcel	High
Express limit (S3)	30.0 (allowed), 30.1 (NOT_ALLOWED)	Line 14 uses > 30; ">= 30" rejects a legal 30 kg express parcel, "> 31" allows illegal ones	High
Started kilogram (S2)	1.0 → 1 kg, 1.1 → 2 kg (also 0.1 → 1 kg)	Loop at line 10: wrong operator or step charges 0 kg or one kg extra on every whole-kg parcel	High
Discount threshold (S4)	9 (no discount), 10 (discount)	Line 17 uses >= 10; "> 10" denies the discount to the 10th-parcel customer	High
Zone letters (S2)	A, B, C, other	Categorical, so no numeric boundary; handled by ECP	—

Forms of BVA and the number of tests

For n numeric variables, normal BVA tests min, min+, nominal, max− and max of each variable while the others stay nominal (4n + 1 tests); robust BVA adds min− and max+ (6n + 1); worst-case BVA takes all combinations of the five values (5^n); robust worst-case takes all combinations of seven (7^n). SwiftShip has two numeric variables, weight and parcels (zone and express are categorical).

Table 7: Cost of each BVA form for SwiftShip

Form	Formula	Tests for n = 2	With zone (3) × express (2)	What it finds / misses
Normal BVA	4n + 1	9	54	Valid edges only; misses 0.0, 50.1 and 30.1 as separate invalid edges
Robust BVA	6n + 1	13	78	Both sides of every edge; assumes one fault at a time
Worst-case BVA	5^n	25	150	Interaction of weight and parcels edges, which the code keeps independent
Robust worst-case BVA	7^n	49	294	Most thorough; most are repetitive
My selection: robust BVA applied per class	edges of W1 and W2 plus parcels edge	**12**	**12**	Every edge in Table 6 on both sides, without repeating each edge for each zone

Standard BVA on the whole range 0.1–50.0 would miss the express limit at 30 kg, because 30 is not an end of the weight range. This is why I apply BVA to each equivalence class (W1: 0.1–30.0, W2: 30.1–50.0) and to the express rule.

Boundary tests with expected results

Unless stated, the tests use zone A (₹40/kg), express = No and parcels = 0 so that only the boundary under test changes.

Table 8: Boundary value test cases

Test	Weight	Express	Parcels	Boundary tested	Expected result	Result on code
B1	0.0	No	0	Lower invalid edge	INVALID	Pass
B2	0.1	No	0	Smallest valid; started kg	1 kg × 40 = 40	Pass
B3	1.0	No	0	Whole kg	1 × 40 = 40	Pass
B4	1.1	No	0	Just over 1 kg	2 × 40 = 80	Pass
B5	30.0	No	0	Upper edge of W1	30 × 40 = 1,200	Pass
B6	30.1	No	0	Lower edge of W2; not express	31 × 40 = 1,240	Pass
B7	50.0	No	0	Upper valid edge	50 × 40 = 2,000	Pass
B8	50.1	No	0	Upper invalid edge	INVALID	Pass
B9	30.0	Yes	0	Express limit allowed	1,200 × 1.5 = 1,800	Pass
B10	30.1	Yes	0	Express limit exceeded	NOT_ALLOWED	Pass
B11	1.0	No	9	Discount threshold − 1	40 (no discount)	Pass
B12	1.0	No	10	Discount threshold	40 × 0.9 = 36	Pass

Recommendation and justification

I recommend **robust BVA applied per class (12 tests, B1–B12)**, with other variables held at nominal values.

- **Risk:** all five boundary faults I seeded in the code (> to >= at lines 1, 14 and 17, and the limit shifted to 31) are killed by B1–B12 (Task 10). Boundary faults are the most likely and most expensive, since one wrong operator affects every parcel at that weight.
- **Cost:** 12 tests against 25 (worst-case) or 49 (robust worst-case); with zone and express the larger forms reach 150–294 tests with almost no new defect-finding power.
- **Why not worst-case BVA:** it is justified when variables interact. In the code, weight and parcels only meet through multiplication (lines 16 and 18) and their conditions are independent, so their edges need not be combined. The real interaction (express with discount) is handled by the decision table in Task 5.
- **Limit:** BVA does not detect the invalid-zone defect or the zone-rate faults, which are covered by ECP.

Task 5 – Apply Other Black-Box Techniques 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other. Do ECP and BVA miss defects caused by combinations of rules?

Evidence Expected: A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

Ans:

Decision Table for Rules S3 and S4

ECP and BVA vary one input at a time. S3 and S4 together produce a result that depends on three conditions at once, so I build a decision table. Conditions: **C1** express requested; **C2** weight ≤ 30 (only meaningful when C1 is true); **C3** parcels ≥ 10. The cause–effect relations are:

NOT_ALLOWED = C1 ∧ ¬C2; express fee = C1 ∧ C2; discount = C3 ∧ ¬(C1 ∧ ¬C2), that is, a rejected request never reaches the discount.

Table 9: Decision table for S3 and S4

	R1	R2	R3	R4	R5	R6
C1 Express requested	No	No	Yes	Yes	Yes	Yes
C2 Weight ≤ 30	–	–	Yes	Yes	No	No
C3 Parcels ≥ 10	No	Yes	No	Yes	No	Yes
Action: NOT_ALLOWED					X	X
Action: add 50 %			X	X		
Action: 10 % discount		X		X		

Table 10: Test cases derived from the decision table

Test	Rule	Weight	Zone	Express	Parcels	Expected result	Result on code
DT1	R1	10.0	B	No	5	10 × 60 = 600	Pass
DT2	R2	10.0	B	No	10	600 × 0.9 = 540	Pass
DT3	R3	10.0	B	Yes	5	600 × 1.5 = 900	Pass
DT4	R4	10.0	B	Yes	10	600 × 1.5 × 0.9 = 810 (not 840)	Pass
DT5	R5	35.0	B	Yes	5	NOT_ALLOWED	Pass
DT6	R6	35.0	B	Yes	10	NOT_ALLOWED (no discount applied)	Pass

DT4 is the key test of how the rules combine: S4 gives 10 % off the **final** charge, so the result is 900 × 0.9 = 810, whereas adding the percentages (+50 % − 10 %) would give 840. Testing express and discount separately cannot tell the two apart.

Comparison with ECP and BVA coverage

Table 11: Which rules the earlier tests already covered

Rule	Covered by ECP (TC1–TC9)	Covered by BVA (B1–B12)	Covered by decision table
R1 No express, no discount	Yes (TC1, TC3)	Yes (B2–B7, B11)	DT1
R2 No express, discount	No	Yes (B12)	DT2
R3 Express, no discount	No	Yes (B9)	DT3
R4 Express and discount	Yes (TC2)	No	DT4
R5 NOT_ALLOWED, no discount	Yes (TC8)	Yes (B10)	DT5
R6 NOT_ALLOWED, discount	No	No	DT6
Rules covered	**3 of 6 (50 %)**	**4 of 6 (67 %)**	**6 of 6 (100 %)**

ECP alone covers half the rule combinations and BVA two-thirds; together they cover five of six (83 %), and the one left out is R6. The table shows two things: ECP/BVA do miss combinations, and most of them are covered incidentally rather than by design (R2 and R3 are covered only because B12 and B9 happen to use the right flags).

Do the missed combinations matter? R6 matters little for the present code, because the return at line 15 happens before the discount at line 17, but it protects against a later change that moves the discount earlier, and DT6 costs one test. R4 matters a great deal: the seeded fault that skips the express charge whenever the customer qualifies for a discount (M14 in Task 10) is not caught by BVA (every BVA test with express uses parcels = 0) nor by the basis-path tests,

and is caught only by TC2 and DT4. This is a typical combination defect that affects the most loyal customers.

Task 6 – Design and Evaluate State-Transition Tests 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel. Which invalid transitions could cost the company money or reputation if they were allowed?

Evidence Expected: A state-transition model with justified valid and invalid transition tests.

Ans:

State-Transition Model of Parcel Tracking

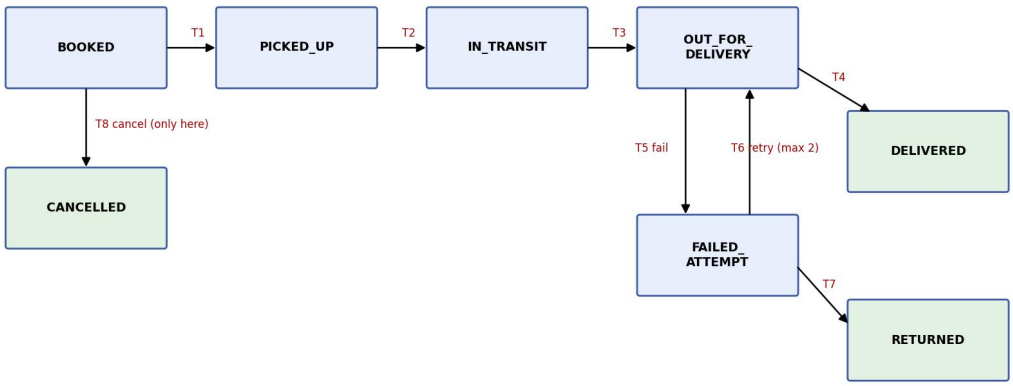


Figure 1: State-transition diagram of parcel tracking (green = end states)

I read "maximum two retries" as one original delivery attempt plus up to two retries, so a parcel can fail at most three times; after the third failed attempt the only legal move is to RETURNED. The specification does not say this explicitly, so it is one of the interpretations I would confirm.

Table 12: State-transition table (blank = not allowed)

Current state	Picked up	Scanned in transit	Out for delivery	Delivered	Delivery fails	Retry	Returned	Cancel
BOOKED	PICKED_UP							CANCELLED
PICKED_UP		IN_TRANSIT						
IN_TRANSIT			OUT_FOR_DELIVERY					
OUT_FOR_DELIVERY				DELIVERED	FAILED_ATTEMPT			
FAILED_ATTEMPT						OUT_FOR_DELIVERY (retry ≤ 2)	RETURNED	
DELIVERED, RETURNED, CANCELLED								

There are 8 states, so 8 × 7 = 56 possible moves between different states: **8 are valid** (T1–T8) and **48 are invalid**. Testing all 48 is not needed because most are blocked by the same guard; I select the ones whose acceptance would cost money or trust.

Tests for valid transitions

Table 13: Valid-transition tests (all 8 transitions covered by 4 tests)

Test	Sequence	Transitions covered	Expected
VT1	BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED	T1, T2, T3, T4	Every move accepted; final state DELIVERED
VT2	OUT_FOR_DELIVERY → FAILED_ATTEMPT → (retry 1) OUT_FOR_DELIVERY → FAILED_ATTEMPT → (retry 2) OUT_FOR_DELIVERY → DELIVERED	T5, T6 twice, T4	Both retries accepted (limit reached but not exceeded)
VT3	OUT_FOR_DELIVERY → FAILED_ATTEMPT → (retry 1) … → FAILED_ATTEMPT → (retry 2) … → FAILED_ATTEMPT (third failure) → RETURNED	T5, T6, T7	Third failure leaves RETURNED as the only move
VT4	BOOKED → CANCELLED	T8	Cancellation accepted before pick-up

Tests for selected invalid transitions

Table 14: Invalid-transition tests with the reason each was selected

Test	Attempted move	Expected	Why it matters (money / reputation)
IT1	BOOKED → DELIVERED	Rejected	Shows "delivered" for a parcel never collected; false proof of delivery and disputes
IT2	IN_TRANSIT → DELIVERED	Rejected	Skips the out-for-delivery scan; allows a parcel lost in transit to be marked delivered
IT3	PICKED_UP → CANCELLED	Rejected	Cancellation rule boundary: refund for a parcel already in the courier's hands (revenue lost, parcel still moving)
IT4	OUT_FOR_DELIVERY → CANCELLED	Rejected	Refund while the courier is at the door; cost incurred without income
IT5	DELIVERED → OUT_FOR_DELIVERY	Rejected	Re-opening a delivered parcel enables false non-delivery claims and refunds
IT6	RETURNED → OUT_FOR_DELIVERY	Rejected	A parcel already sent back to the sender is sent out again; double handling cost
IT7	CANCELLED → PICKED_UP	Rejected	Courier collects a cancelled booking after the customer was refunded
IT8	FAILED_ATTEMPT → DELIVERED	Rejected	Delivery recorded without a new attempt; no proof of delivery
IT9	FAILED_ATTEMPT (after retry 2 failed) → OUT_FOR_DELIVERY	Rejected	Retry limit exceeded: unlimited delivery attempts at the company's expense

Selection logic. I picked at least one test for each end state (IT5, IT6, IT7), for each of the two rules the specification states (IT3, IT4 for cancellation; IT9 for retries), and for the two most damaging "skips" that create false delivery records (IT1, IT2, IT8). The unselected moves, such as OUT_FOR_DELIVERY → IN_TRANSIT, would confuse customers but cost little. In total the state suite has **4 valid + 9 invalid = 13 tests**.

Rules specifically tested. Maximum two retries: VT2 (two retries allowed), VT3 (third failure forces RETURNED) and IT9 (third retry refused). Cancellation: VT4 (allowed from BOOKED) and IT3, IT4 (refused after pick-up), which is the boundary of the rule.

Task 7 – Construct the Control Flow Graph **4 Marks**

CO2 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph. Does the structure of the code match the structure of the specification?

Evidence Expected: A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

Ans:

Control Flow Graph of shipCharge

I group consecutive statements without branching into one node (a basic block). A decision statement ends a node, and each "return" leads to the Exit node. Yellow nodes are decisions.

Table 15: Mapping of nodes to line numbers

Node	Line(s)	Statement	Type
N1	1	if weight <= 0 or weight > 50	Decision (compound condition)
N2	2	return INVALID	Return
N3	3, 4	rate = 40; if zone == "B"	Decision
N4	5	rate = 60	Process
N5	6	else if zone == "C"	Decision
N6	7	rate = 90	Process
N7	8, 9	charge = 0; w = weight	Process
N8	10	while w > 0	Decision (loop test)
N9	11, 12	charge = charge + rate; w = w − 1	Process (loop body)
N10	13	if express	Decision
N11	14	if weight > 30	Decision
N12	15	return NOT_ALLOWED	Return
N13	16	charge = charge × 1.5	Process
N14	17	if parcels >= 10	Decision
N15	18	charge = charge × 0.9	Process
N16	19	return charge	Return
Exit	–	End of function	Exit

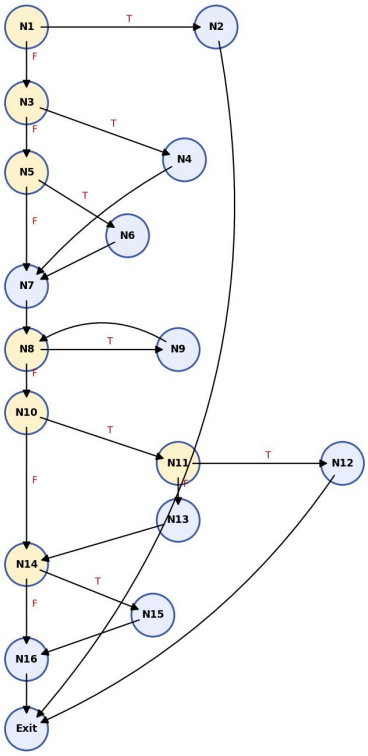


Figure 2: Control flow graph of shipCharge (T = condition true, F = condition false)

Edges (23): N1→N2 (T), N1→N3 (F), N2→Exit, N3→N4 (T), N3→N5 (F), N4→N7, N5→N6 (T), N5→N7 (F), N6→N7, N7→N8, N8→N9 (T), N8→N10 (F), N9→N8 (back edge of the loop), N10→N11 (T), N10→N14 (F), N11→N12 (T), N11→N13 (F), N12→Exit, N13→N14, N14→N15 (T), N14→N16 (F), N15→N16, N16→Exit. There are 17 nodes and 7 decision nodes (N1, N3, N5, N8, N10, N11, N14).

Decision nodes, the loop and the compound condition

- **The loop (N8–N9):** N9 returns to N8; the number of iterations is the number of started kilograms (1 for 0.1–1.0, up to 50 for 50.0). The loop cannot execute zero times, because N1 has already rejected weight ≤ 0.
- **The compound condition at line 1:** "weight <= 0 or weight > 50" is one decision node but two conditions. With short-circuit evaluation, weight > 50 is evaluated only when weight <= 0 is false. If it is expanded, the graph has an extra node and 25 edges (18 nodes), which raises the complexity from 8 to 9 (Task 8).
- **N11 tests `weight`, not `w`:** the loop has changed `w`, so the code correctly keeps the original in `weight`. Using `w` here would always be ≤ 0 and would never reject express parcels.

Does the code structure match the specification?

Table 16: Specification rules compared with code structure

Rule	Specification structure	Code structure	Match?
S1	One range check with two edges (0 and 50)	N1: one decision, two conditions, return INVALID (N2)	Yes
S2 zone	Four outcomes: A, B, C, or INVALID	N3, N5: three outcomes only; default rate 40 (Zone A) with no INVALID path	**No: missing branch**

Rule	Specification structure	Code structure	Match?
S2 started kg	Round up the weight to a whole kilogram	Loop N8–N9 subtracting 1 until w ≤ 0	Different in form; equal in result for one-decimal weights
S3	Express adds 50 %; reject above 30 kg	N10, N11, N12, N13	Yes (but the rejection comes after the loop has run)
S4	10 % off the final charge	N14, N15 after the express charge	Yes
Negative parcels	Not specified	Treated as "no discount"	Specification gap

The most important mismatch is the **missing INVALID path for zones**. The specification implies a four-way choice (A, B, C, other), but the code is a two-step if/else-if that treats "A" as the default, so every unknown value falls into Zone A. This is a defect that the graph itself shows: there is no edge from N3/N5 to N2 or any other rejection. Both versions have seven decisions (the specification has: range, three zone tests, express, 30 kg, discount), so the cyclomatic complexity looks right; but in the code the seventh decision is the loop, and the zone-validity decision is absent. Equal complexity does not mean equal structure.

Task 8 – Evaluate Path Testing and Cyclomatic Complexity 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code. How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Evidence Expected: A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

Ans:

Cyclomatic Complexity of shipCharge

Table 17: Cyclomatic complexity V(G) calculated by three methods

Method	Derivation	Result
Edges and nodes	V(G) = E − N + 2P = 23 − 17 + 2(1)	8
Predicate nodes	V(G) = P + 1 = 7 decision nodes + 1 (N1, N3, N5, N8, N10, N11, N14)	8
Regions	Bounded regions (the loop, and each decision's enclosed area) 7 + 1 outer region	8
Compound condition expanded (line 1 as two conditions)	E = 25, N = 18: 25 − 18 + 2 = 9 (predicates 8 + 1)	9

The three methods agree on V(G) = 8, so **at least 8 tests** are needed to cover all independent paths; if the two conditions on line 1 are counted separately the figure is 9.

Basis (independent) paths and test data

I take the simplest path as the baseline (P1) and flip one decision at a time to get the other seven. The baseline loops once.

Table 18: Basis paths, test data and expected results

Path	Nodes visited	Decision flipped	Test data (weight, zone, express, parcels)	Expected
P1	N1-N3-N5-N7-N8-N9-N8-N10-N14-N16-Exit	Baseline	1.0, A, No, 0	40

Path	Nodes visited	Decision flipped	Test data (weight, zone, express, parcels)	Expected
P2	N1-N2-Exit	N1	0.0, A, No, 0	INVALID
P3	N1-N3-N4-N7-N8-N9-N8-N10-N14-N16-Exit	N3 (zone B)	1.0, B, No, 0	60
P4	N1-N3-N5-N6-N7-N8-N9-N8-N10-N14-N16-Exit	N5 (zone C)	1.0, C, No, 0	90
P5	N1-N3-N5-N7-N8-N9-N8-N9-N8-N10-N14-N16-Exit	N8 (loop repeats)	2.0, A, No, 0	80
P6	N1-N3-N5-N7-N8-N9-N8-N10-N11-N13-N14-N16-Exit	N10 (express), N11 false	1.0, A, Yes, 0	60
P7	N1-N3-N5-N7-N8-(N9-N8)×35-N10-N11-N12-Exit	N11 true	35.0, A, Yes, 0	NOT_ALLOWED
P8	N1-N3-N5-N7-N8-N9-N8-N10-N14-N15-N16-Exit	N14	1.0, A, No, 10	36

All eight paths are feasible and all pass on the pseudocode. P7 shows a data dependency: to make N11 true the weight must exceed 30, so the loop necessarily runs at least 31 times, not once.

Infeasible paths

Table 19: Paths that cannot be executed

Path	Why it is infeasible
N7 → N8 (false) → N10: loop skipped	It needs w = weight ≤ 0, but N1 has already returned INVALID for weight ≤ 0
Express with weight > 30 but only one loop iteration (N11 true, N8 once)	weight > 30 forces at least 31 iterations
Express allowed (N11 false) after 31 or more iterations	Iterations ≥ 31 mean weight > 30, which makes N11 true
N1 true by both conditions at once (weight ≤ 0 and weight > 50)	A weight cannot be both

Effect of the loop, and how to test it

Because the loop runs from 1 to 50 times, the number of feasible complete paths is far above 8. Counting by zone (3), number of kilograms (1–50), express (no/yes, with express impossible above 30 kg) and discount (no/yes): 30 weights × 4 combinations + 20 weights × 3 (two non-express, plus NOT_ALLOWED) = 180 per zone, so 3 × 180 + 1 invalid path = **541 feasible paths**. Exhaustive path testing is not realistic, so I use loop-testing rules on top of the basis set.

Table 20: Loop tests (zone A, express No, parcels 0)

Iterations	Weight	Expected	Remark
0 (skip)	—	—	Infeasible, N1 rejects weight ≤ 0 (Table 19)
1	0.1	40	Minimum; also checks started-kilogram rounding
2	1.1	80	Second pass; fractional weight
m = 25 (typical)	25.0	1,000	Typical many-iteration run
n − 1 = 49	49.0	1,960	One below the maximum
n = 50 (maximum)	50.0	2,000	Maximum loop count
n + 1 = 51	50.1	INVALID	The loop is never reached; line 1 rejects it

Is 8 enough? No. Eight tests give each independent path once but give no assurance on the 541 paths. In Task 10 the eight basis tests find only 8 of the 15 defects I examined; adding the loop

tests brings this to 11. The main gaps are data-dependent: boundary values (the basis data use weight 35, not 30.1), combinations (express together with discount), and the omitted zone check, which has no path to test.

Task 9 – Judge Test Adequacy and Coverage 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is "enough". Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Evidence Expected: A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

Ans:

Coverage Achieved by the Basis-Path Tests

I measured the eight basis-path tests (P1–P8, Table 18) against the criteria. The pseudocode has 19 statements, 7 decisions (14 branch outcomes), and 8 atomic conditions: weight <= 0, weight > 50, zone == B, zone == C, w > 0, express, weight > 30 and parcels >= 10 (16 outcomes).

Table 21: Coverage criteria for shipCharge and what the tests achieve

Criterion	What it requires	Elements	Covered by P1–P8	Coverage	Smallest suite
Statement	Every line executed	19	19	100 %	3 tests
Branch (decision)	Each decision both true and false	14	14	100 %	4 tests
Condition	Each atomic condition both true and false	16	15 (weight > 50 never true)	93.8 %	5 tests
Multiple condition	All feasible condition combinations	15 (line 1 has 3 feasible of 4)	14	93.3 %	5 tests
All paths	Every feasible path	541	8	1.5 %	Not practical

The basis tests reach full statement and branch coverage but not condition coverage: with weight 0 as the only invalid weight in P2, the second condition "weight > 50" is never true (because of short-circuit evaluation, it is not even evaluated). **One more test, weight 50.1, closes the gap** and gives 100 % condition and multiple-condition coverage (and MC/DC for line 1, which needs n + 1 = 3 tests: weight 0, 50.1 and an in-range weight). There is no other compound condition in the module, so these criteria cost one extra test and no more.

Why statement coverage is not enough

Three tests (weight 0; weight 10, zone B, express, parcels 12; weight 35, zone C, express) execute all 19 statements, so a report would read "100 % coverage". But the same three tests cover only 11 of 14 branches (78.6 %) and 12 of 16 conditions (75 %): they never run a Zone A parcel (N5 false), never run without express (N10 false), and never run without a discount (N14 false). That is exactly the common case: most of the 40,000 daily parcels are ordinary, non-express, non-discounted bookings. A single statement-coverage number would give management false assurance.

Would 100 % coverage have found the invalid-zone defect?

No. Zone "D" follows the same path as Zone A (N3 false → N5 false → N7). The missing INVALID return is an omission, and structural coverage measures only the code that exists. Even the all-paths criterion (541 paths) would be satisfied without a test using an unlisted zone. In my own

tests the ECP set reaches 100 % statement, branch and condition coverage, yet it finds the defect only because TC6 and TC7 were designed from the specification. Coverage tells me what has not been tested, never what is missing.

Recommended criterion

I recommend **100 % branch coverage plus 100 % condition coverage (condition/decision coverage), with MC/DC applied to the compound condition at line 1, together with specification-based coverage: every equivalence class, every boundary and every decision-table rule.**

- **Risk:** the module computes money, so every decision, and every outcome of every condition, must be exercised; statement coverage leaves the ordinary cases untested (above).
- **Cost:** the minimum suite for this criterion is 5 tests, and the module has one compound condition, so the extra cost over branch coverage is one test.
- **Why not all-paths or full path coverage:** 541 paths for a 19-line function would cost more than the risk reduction justifies, and the paths that differ only in loop count add nothing the loop tests do not cover.
- **Why the single number is not enough:** it must be reported with the specification coverage, since structural coverage cannot detect omissions.

Task 10 – Compare Black-Box and White-Box Effectiveness 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9). Which approach actually found more of the important defects in shipCharge?

Evidence Expected: A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

Ans:

Black-Box and White-Box Effectiveness

The pseudocode contains only **one** real defect (the missing zone check, D0). One defect is too few to compare approaches, so I also seeded 14 plausible faults, one at a time (mutation testing), and ran each test set against each faulty version. A fault counts as found when the faulty version gives a result different from the original on at least one test. These are my own seeded faults, so the comparison indicates relative strength and is not a measure of field defects.

Table 22: Defects examined (D0 is real; M1–M14 are seeded)

ID	Defect	Kind
D0	Invalid zone accepted and charged as Zone A (line 3–7 has no INVALID return)	Omission
M1	Line 1: weight <= 0 becomes weight < 0	Boundary
M2	Line 1: weight > 50 becomes weight >= 50	Boundary
M3	Line 14: weight > 30 becomes weight >= 30	Boundary
M4	Line 14: weight > 30 becomes weight > 31	Boundary
M5	Line 17: parcels >= 10 becomes parcels > 10	Boundary
M6	Line 10: while w > 0 becomes while w >= 0 (extra kilogram for whole weights)	Loop
M7	Line 16: factor 1.5 becomes 1.15	Calculation
M8	Line 18: factor 0.9 becomes 0.8	Calculation

ID	Defect	Kind
M9	Line 5: Zone B rate 60 becomes 65	Calculation
M10	Line 7: Zone C rate 90 becomes 80	Calculation
M11	Line 10: while w > 0 becomes while w > 0.5 (part kilograms not charged)	Loop
M12	Line 12: w = w − 1 becomes w = w − 2	Loop
M13	NOT_ALLOWED check applied to every parcel over 30 kg, not just express	Misplaced decision
M14	Express charge skipped when the customer has 10 or more parcels	Rule interaction

Table 23: Defects found by each test set (✓ = found)

Defect	ECP (9)	BVA (12)	Decision table (6)	Basis paths (8)	Loop tests (6)	Black-box all (26)	White-box all (14)
D0	✓					✓	
M1		✓		✓		✓	✓
M2		✓			✓	✓	✓
M3		✓				✓	
M4		✓				✓	
M5		✓	✓	✓		✓	✓
M6	✓	✓	✓	✓	✓	✓	✓
M7	✓	✓	✓	✓		✓	✓
M8	✓	✓	✓	✓		✓	✓
M9	✓		✓	✓		✓	✓
M10	✓			✓		✓	✓
M11	✓	✓			✓	✓	✓
M12	✓	✓	✓	✓	✓	✓	✓
M13	✓	✓			✓	✓	✓
M14	✓		✓			✓	
Found (of 15)	**10**	**11**	**7**	**8**	**5**	**15**	**11**

(ECP counts TC1–TC9; TC9 ran on the code but has no firm expected value, so it is not used as a pass/fail test in the kill count. Black-box all = ECP + BVA + decision table = 26 tests; white-box all = basis paths + loop tests = 14 tests.)

What each approach found

Table 24: Defects found by black-box only, white-box only and both

Group	Defects	Count	Share of 15
Black-box only	D0, M3, M4, M14	4	26.7 %
White-box only	none	0	0 %
Found by both	M1, M2, M5–M13	11	73.3 %
Found by neither	none	0	0 %

Why each approach missed what it missed

- **White-box missed D0:** the missing INVALID return leaves no code to cover, and zone “D” takes the same path as zone A.

- **White-box missed M3 and M4:** path testing picks any data that drives a path, and my basis tests use weight 35 (not 30.0 or 30.1) to make N11 true, so a limit moved from 30 to 31 is unnoticed. Branch coverage is satisfied by the wrong test data.
- **White-box missed M14:** express (P6) and discount (P8) are each executed on separate paths, but never together, so their interaction is untested although all 8 paths and 100 % branch coverage are achieved.
- **Black-box techniques each missed something:** ECP missed all boundary faults (M1–M5); BVA missed zone defects (D0, M9, M10) because it uses zone A only; the decision table used zone B and weights 10/35 and missed boundaries. The combination found everything.

Interpretation. In shipCharge black-box testing found more important defects (15 of 15 against 11 of 15), and white-box added no unique defect. This is because the code is a close transcription of a short specification, so almost every fault is a deviation from the specification and a test designed from it exposes it. White-box work was not wasted: it measured the completeness of the black-box suite (Task 9), exposed the gap in "weight > 50" condition coverage, and showed that the loop cannot run zero times. Per test, white-box was slightly more economical (11 defects with 14 tests, 0.79 per test, against 15 with 26, 0.58 per test), but covers less.

Overlapping tests that can be removed

The union of all sets has 40 test entries, of which 32 are distinct. A set of 15 carefully chosen tests (Table 25) still finds all 15 defects and reaches 100 % statement, branch, condition and multiple-condition coverage, 6 of 6 decision-table rules and every boundary of Table 6. So **25 of the 40 entries (63 %) can be removed without loss of defect-finding power**: for example, B3 (1.0 kg) is repeated by P1; TC4 and TC5 (−2.0, 60.0) duplicate B1 and B8; DT1, DT3 and DT5 are covered by tests with the same rule and boundary data; and TC1 and P5 are subsumed by B4 and the loop tests.

Would this result hold for other modules?

Not necessarily. For the **tracking module**, state-transition tests (black-box) are the only practical way to find illegal transitions, so black-box would again lead. For a module whose code goes beyond its specification (caching, special cases, optimisations, error handling not in the requirements, or complex loops and pointers), white-box tests would find faults that no specification-derived test aims at. The result here follows from the module's properties: small, specification-driven, with defects of omission and boundary type. Besides, 15 defects of my own seeding is a small sample, so I would repeat the comparison with real defect history before generalising.

Task 11 – Recommend and Defend the Final Test Suite — 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Evidence Expected: A technically justified final test suite supported by black-box, white-box and coverage evidence.

Ans:

Final Regression Test Suite

I built the suite from the union of all techniques, keeping each test that adds a defect, boundary, rule or coverage element that nothing else supplies. It has **15 calculation tests and 13 tracking tests, 28 in all**.

Table 25: Final regression suite for the calculation module (SS01–SS15)

Test	Weight	Zone	Express	Parcels	Expected	Technique / purpose
SS01	0.0	A	No	0	INVALID	BVA: lower invalid edge
SS02	0.1	A	No	0	40	BVA, loop (1 iteration): smallest valid, started kg
SS03	1.1	A	No	0	80	BVA, loop (2 iterations): part-kg rounding
SS04	30.1	A	No	0	1,240	BVA/ECP W2: heavy parcel without express
SS05	50.0	A	No	0	2,000	BVA, loop (50 iterations): upper valid edge
SS06	50.1	A	No	0	INVALID	BVA, condition coverage (weight > 50 true)
SS07	10.0	B	No	9	600	ECP Zone B; DT R1; parcels edge 9
SS08	40.0	C	No	0	3,600	ECP Zone C; W2 valid
SS09	5.0	D	No	0	INVALID	ECP invalid zone (exposes D0)
SS10	5.0	(blank)	No	0	INVALID	ECP missing zone (exposes D0)
SS11	10.0	B	No	10	540	DT R2; discount edge 10
SS12	30.0	B	Yes	0	2,700	DT R3; BVA express limit allowed (1,800 × 1.5)
SS13	10.0	B	Yes	10	810	DT R4: express and discount together
SS14	30.1	C	Yes	0	NOT_ALLOWED	DT R5; BVA express limit exceeded
SS15	35.0	A	Yes	12	NOT_ALLOWED	DT R6: rejected request is not discounted

Tracking tests (13): VT1–VT4 (Table 13) and IT1–IT9 (Table 14), model-based state-transition tests covering all 8 valid transitions and the 9 most costly invalid ones, including both stated rules (retry limit, cancellation).

Evidence for the suite

Table 26: Final suite compared with the alternatives

Suite	Calculation tests	Defects found (of 15)	Branch / condition coverage	Rules R1–R6	Calculation run time (assumed)	Verdict
ECP only	9	10	100 % / 100 %	3	< 1 min	Misses all boundary faults
BVA only	12	11	86 % / 88 %	4	< 1 min	Misses zones and combinations
Basis paths only	8	8	100 % / 94 %	4	< 1 min	Misses boundaries, zone check
Union of everything	40 (32 distinct)	15	100 % / 100 %	6	≈ 1 min	Redundant
Exhaustive worst-case BVA (weight × zone × express × parcels)	400	15 (expected)	100 % / 100 %	6	≈ 13 min	Costly for no new defects
Recommended	**15**	**15**	**100 % / 100 %**	**6**	**≈ 0.5 min**	**Smallest suite with full evidence**

Run time assumes 2 s per calculation test (a call to the charge service, with set-up and comparison) and 15 s per tracking test (each creates a parcel and moves it through its states). On that basis the whole suite takes 15 × 2 + 13 × 15 = **225 s, about 3.8 minutes**.

Coverage achieved and rule combinations

- **Structural:** 100 % statement (19/19), 100 % branch (14/14), 100 % condition (16/16) and all 15 feasible multiple-condition combinations; V(G) = 8 basis paths all exercised.
- **Specification:** all 17 equivalence classes except the negative-parcel class (P3, awaiting a rule); all boundaries of Table 6; 6 of 6 decision-table rules.
- **Tracking:** 8 of 8 valid transitions; 9 of 48 invalid transitions chosen by risk.
- **Defects:** all 15 (D0 and the 14 seeded faults).

Why this suite is better than the alternatives

It is smaller than the union (15 against 40) but loses no defect or coverage, and it is far cheaper than exhaustive combination (15 against 400). Single-technique suites each miss between 4 and 8 defects. The largest suite is therefore not the best: SS01–SS15 reuse each test for several purposes (for example SS12 is simultaneously a Zone B rate check, a boundary test and decision-table rule R3).

Known gaps and risks

- **Open defect D0:** SS09 and SS10 will fail until the zone check is added; the suite is “red” by design until the fix.
- **Specification gaps:** negative parcels (TC9), weights with more than one decimal place, lower-case zone letters, and rounding of the charge are not defined, so no test has an expected result for them. They should be settled with the business owner.
- **Floating-point:** loop subtraction and the 1.5 and 0.9 factors are checked with a small tolerance; a real implementation using decimal arithmetic should be tested separately.
- **Tracking module:** only model-based tests exist, since no tracking code is available for white-box analysis; most invalid transitions (39 of 48) are untested.
- **Seeded faults are my own choice:** the 15/15 result shows the suite is strong against these faults, not that no others exist; non-functional (load, security) and integration tests are outside this suite.

DAY-4 DESIGN CHALLENGE

Task 12 – Re-evaluate and Adapt the Test Suite 8 Marks

CO2 | Bloom's Level: K5 – Evaluate

Changed Requirement: SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds ₹50 per parcel. The developers have changed shipCharge by adding two new decisions after line 16. The suite must still run within 30 minutes before every release. Re-evaluate your original solution in response to these changed conditions. You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis

CO2 | Bloom's Level: K5 – Evaluate *2 Marks*

Evidence Expected: A clear analysis of how the changes affect the original test design.

Ans:

Impact of the Changes

The change request does not say where the new rules sit in the calculation. I assume the natural reading of "after line 16": both decisions follow the express block (not inside it) and come before the discount, so that S4 still discounts the "final charge". These assumptions are themselves test targets (Part B).

```
13  if express then
14      if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
16a if zone == "C" then
16b     charge = charge * 1.08        (new S5: fuel surcharge 8 %)
16c if weekend then
16d     charge = charge + 50          (new S6: weekend delivery)
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Table 27: What has changed in the specification and in the code

Item	Before	After
Specification	S1–S4	S5: Zone C charge × 1.08 (8 % fuel surcharge); S6: weekend delivery adds ₹50 per parcel
Function inputs	weight, zone, express, parcels	Also a fifth input, weekend
Code	19 lines, 7 decisions	Two new decisions (16a, 16c) and two new actions (16b, 16d)
Order of calculation	Rate × kg → express → discount	Rate × kg → express → fuel surcharge → weekend fee → discount (so S4 applies to the surcharge and the weekend fee)
Constraint	None	Whole suite must run in 30 minutes before every release

Which test design elements are affected

Table 28: Effect on classes, boundaries, decision table, control flow graph and complexity

Test design element	Before	After	Effect
Equivalence classes	17 classes	19 classes: new weekend true (WK1) and false (WK2); Zone C splits into "with surcharge"	Zone C expected values change; weekend is a new input

Test design element	Before	After	Effect
Boundaries	Weight 0/0.1/30/30.1/50/50.1; parcels 9/10	Unchanged. The new rules have no numeric edge; the constants 1.08 and 50 are checked by value	Boundary tests stay valid; Zone C ones need new expected values
Decision table	3 conditions, 6 rules	5 conditions (express, weight ≤ 30, zone C, weekend, discount): 16 valid combinations + 2 NOT_ALLOWED = **18 rules**	Rules triple; order of surcharge, weekend fee and discount matters
Control flow graph	17 nodes, 23 edges	21 nodes, 29 edges (Figure 3)	Two decisions added
Cyclomatic complexity	V(G) = 8	V(G) = 29 − 21 + 2 = **10**; predicates 9 + 1 = 10	Two more basis paths
Feasible paths	541	**1,021** (340 per zone × 3 + 1 invalid)	Exhaustive path testing even less feasible

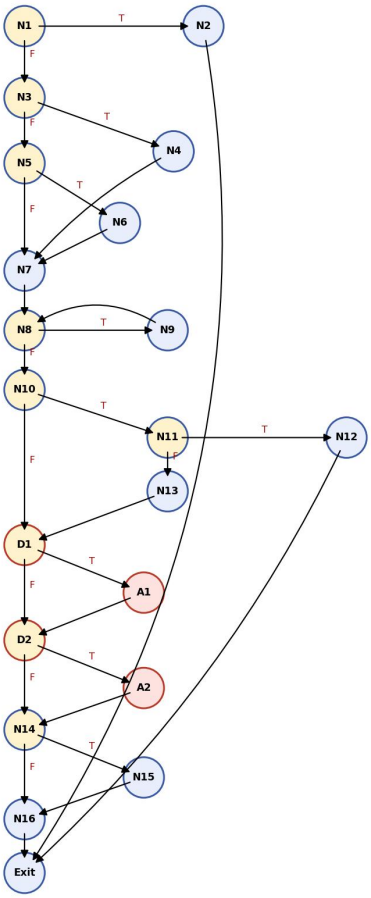


Figure 3: Control flow graph after the change (red = new nodes D1 = line 16a, A1 = 16b, D2 = 16c, A2 = 16d)

The new decision at line 16a tests the same condition as line 6 (zone == "C"). The two decisions are **correlated**: Zone C always takes both true branches, other zones both false. A path with line 6 true and line 16a false cannot be executed, so the structural complexity of 10 overstates the real variety. Finding 10 independent feasible paths needs a path that follows the NOT_ALLOWED route for a non-C zone (the tenth path in the list after Table 31). I would suggest to the developers that they read the zone surcharge from the same rate table as the per-kilogram rate; this would remove the duplicated decision and one source of defects.

Why the 30-minute limit matters

The suite runs before every release, so a slow suite tends to be skipped or trimmed. The new rules multiply the combinations that could be tested: with weight (10 boundary values), zone (4), express (2), weekend (2) and parcels (5 values) an exhaustive worst-case set would have **10 × 4 × 2 × 2 × 5 = 800 tests**, or 1,600 s on my timing assumption (2 s each), and 1,795 s with the 13 tracking tests: 29.9 minutes, leaving no margin. The limit therefore forces selection by technique rather than enumeration.

B. Evaluation of the Original Suite

CO2 | Bloom's Level: K5 – Evaluate *2 Marks*

Evidence Expected: A technically supported evaluation of the original suite under the new conditions.

Ans:

Is the Final Suite Still Adequate?

Table 29: Status of each original test after the change

Group	Tests	Status	Reason
Calculation tests not involving Zone C	SS01–SS07, SS09–SS13, SS15 (13)	Still valid	Same inputs and results with weekend = No; only the new fifth input must be added to each call
Zone C test with a result	SS08 (40 kg, Zone C)	**New expected result**	3,600 becomes 3,600 × 1.08 = 3,888
Zone C test that is rejected	SS14 (30.1 kg, Zone C, express)	Still valid	NOT_ALLOWED is returned at line 15, before the new decisions
Tracking tests	VT1–VT4, IT1–IT9 (13)	Still valid	Parcel states are unchanged by the new charge rules
Obsolete tests	—	**None**	No rule has been withdrawn

So of the 15 calculation tests, 14 stay valid and 1 needs a new expected value; none is obsolete. All of them need a harness change for the fifth parameter.

What the old suite no longer covers

Table 30: Old suite run against the changed code

Measure	Old suite on new code	Gap
Branch coverage	17 of 18 outcomes (94.4 %)	Weekend true (line 16c) never taken
Condition coverage	19 of 20 outcomes (95.0 %)	Weekend true never taken
Decision-table rules	7 of 18 (38.9 %)	11 valid combinations involving Zone C or weekend are untested
New defects found (6 seeded)	3 of 6	Missed: weekend fee added after the discount (N2), weekend fee ignored for express (N5), surcharge skipped when a discount applies (N6)

The seeded changes were: **N1** surcharge and weekend fee placed inside the express block; **N2** weekend fee added after the discount; **N3** surcharge also charged to Zone B; **N4** factor 1.8 instead of 1.08; **N5** weekend fee ignored for express parcels; **N6** surcharge skipped when the discount applies. The old suite finds N1, N3 and N4 only because SS08 and the Zone B tests happen to touch Zone C; it misses the three order and interaction faults, which are the ones most likely to cause undercharging.

Coverage criterion and the original assumptions

- **Branch and condition coverage can still be met:** one test with weekend = Yes closes the single gap. The minimum suite for the criterion rises from 5 to 6 tests.
- **New paths and combinations not covered:** weekend with discount (is the ₹50 discounted? my code gives (charge + 50) × 0.9), Zone C with express (is 8 % added to the express-inclusive charge?), and all three together. The specification does not state these orders; the tests will fix them, and the business owner must confirm them.
- **Placement risk:** if the new decisions were written inside the express block, the surcharge and weekend fee would be skipped for ordinary parcels, which is the most likely mistake given "after line 16"; the non-express Zone C and weekend tests catch it (N1).

C. Adaptation or Redesign

CO2 | Bloom's Level: K5 – Evaluate *2 Marks*

Evidence Expected: An appropriate test-design response to the changed requirement with clear reasoning.

Ans:

Decision: Modify the Test Suite

I **MODIFY** the suite. I do not retain it unchanged: it covers only 7 of 18 rules and misses three of the six new faults. I do not redesign it: 14 of 15 calculation tests and all 13 tracking tests are still correct, the boundary and zone design is unchanged, and the techniques are still the right ones. The change is an extension of the decision table and a few expected values.

New and changed test cases

The decision table now has four binary factors (express, Zone C, weekend, discount): 16 combinations. Calculation tests are cheap, so I test the full factorial rather than a pairwise subset; because the weekend fee is added and the discount is multiplied, the order of operations matters and a sample could miss it. Five combinations are already covered by SS07, SS08, SS11, SS12 and SS13; **11 new combinations plus one extra NOT_ALLOWED test** are added, with weight 10.0 (so that Zone C is 900, Zone A is 400).

Table 31: Changed test (SS08) and new tests (SS16–SS27) with expected results

Test	Weight	Zone	Express	Parcels	Weekend	Expected result	Technique
SS08*	40.0	C	No	0	No	3,600 × 1.08 = **3,888**	ECP (changed)
SS16	10.0	A	No	0	Yes	400 + 50 = 450	DT: weekend only
SS17	10.0	A	No	10	Yes	(400 + 50) × 0.9 = 405	DT: order of fee and discount
SS18	10.0	C	No	10	No	900 × 1.08 × 0.9 = 874.8	DT: surcharge and discount
SS19	10.0	C	No	0	Yes	900 × 1.08 + 50 = 1,022	DT: surcharge and weekend
SS20	10.0	C	No	10	Yes	(972 + 50) × 0.9 = 919.8	DT
SS21	10.0	A	Yes	0	Yes	400 × 1.5 + 50 = 650	DT: express and weekend
SS22	10.0	A	Yes	10	Yes	(600 + 50) × 0.9 = 585	DT
SS23	10.0	C	Yes	0	No	900 × 1.5 × 1.08 = 1,458	DT: express and surcharge
SS24	10.0	C	Yes	10	No	1,458 × 0.9 = 1,312.2	DT

Test	Weight	Zone	Express	Parcels	Weekend	Expected result	Technique
SS25	10.0	C	Yes	0	Yes	1,458 + 50 = 1,508	DT
SS26	10.0	C	Yes	10	Yes	(1,458 + 50) × 0.9 = 1,357.2	DT: all factors
SS27	35.0	C	Yes	0	Yes	NOT_ALLOWED	DT: rejection unaffected by new fees

All 12 new expected results come from the rules S1–S6 (not from the code) and agree with the changed code. Seeded against these tests, the modified suite finds **6 of 6** new faults (old suite: 3 of 6). The ten basis paths for the new graph (data below; the modified suite reaches 100 % branch coverage, so every decision outcome on them is exercised) are: Zone C baseline (1.0 kg → 97.2); INVALID; Zone B (60); Zone A (40); two iterations (194.4); express (145.8); NOT_ALLOWED for Zone C (SS27); weekend (147.2); discount (87.48); NOT_ALLOWED for Zone A (SS15).

Effect on coverage and number of tests

Table 32: Suite size, coverage and run time before and after the change

Measure	Original suite	Modified suite
Calculation tests	15	27 (14 unchanged + 1 changed + 12 new)
Tracking tests	13	13
Total tests	28	**40**
Branch / condition coverage on the current code	100 % / 100 % (14/14, 16/16)	100 % / 100 % (18/18, 20/20)
Decision-table rules covered	6 of 6	18 of 18
Feasible basis paths covered	8 of 8	10 of 10
Seeded defects found	15 of 15 (old code)	15 of 15 (old) and 6 of 6 (new)
Run time (2 s per calculation test; 15 s per tracking test)	30 + 195 = 225 s (3.8 min)	54 + 195 = 249 s (4.2 min)
Share of the 30-minute limit	12.5 %	13.8 %

Which techniques derive the changes

- **Decision table / combination testing** for SS16–SS27 (the order and interaction of surcharge, weekend fee, express and discount).
- **ECP** for the changed Zone C expected value (SS08) and for the weekend input classes.
- **White-box analysis** (new CFG and basis paths) to show that weekend = Yes was the one branch not executed and that the zone decision is duplicated.
- **BVA** needs no new tests; the existing edges remain valid.

D. Evidence-Based Defence

CO2 | Bloom's Level: K5 – Evaluate *2 Marks*

Evidence Expected: A well-supported final test suite integrating test design evidence and the changed constraints.

Ans:

Defending the Modified Suite

Table 33: Evidence supporting the decision to modify the suite

Evidence	What it shows	Effect on my decision

Evidence	What it shows	Effect on my decision
Equivalence classes and boundaries	17 classes and 8 boundary points are unchanged; only Zone C expected values change (SS08: 3,888)	Keep the ECP/BVA tests (14 of 15 valid)
Decision-table combinations	Rules grow from 6 to 18; the old suite covers 7 (38.9 %) and misses 3 of 6 new faults, including the ones about order (N2, N5, N6)	Extend the table: 11 combinations + 1 rejection test
Cyclomatic complexity and basis paths	V(G) rises from 8 to 10; feasible paths from 541 to 1,021; zone decision duplicated, so one flip is infeasible	Add the two basis-path conditions through the tests above; do not attempt all paths
Coverage achieved	Old suite reaches 17/18 branches and 19/20 conditions; the modified suite 18/18 and 20/20	The adequacy criterion (branch + condition + rules) is still reachable
Execution time limit	Modified suite runs in about 4.2 minutes (13.8 % of 30); exhaustive combination would take 29.9 minutes	Selected combinations fit with a wide margin; a redesign towards larger sets would not
Risk	Money module, 40,000 parcels a day; one wrong order of operations misprices every weekend Zone C parcel with a discount	Test every combination of the four binary factors because it is cheap

Reasoning. Retaining the old suite would leave 11 of 18 rules and half the new fault types untested on a function that prices every parcel. A full redesign would discard tests that are still correct and a classification of boundaries that is unchanged. Modification adds 12 tests, changes 1, and uses 13.8 % of the time budget, which keeps 86 % for later growth. The suite also remains small enough to run before every release, which is the real constraint: the 30-minute limit is not a reason to cut tests now, but it is a reason to avoid growing the suite by enumeration later.

Remaining risks and monitoring. (1) The order of surcharge, weekend fee and discount rests on my reading of “after line 16” and “final charge”; the business owner should confirm it, and if it differs only the expected values of SS17, SS20, SS22 and SS26 change. (2) The invalid-zone defect D0 is still open, so SS09 and SS10 will fail until the fix. (3) Run times are assumptions; I would record the measured times of the first three releases and cut or tier tests if the suite passes 15 minutes. (4) The tracking module may need new tests if weekend delivery changes its states, which the change request does not say.

MARK DISTRIBUTION

Task	Assessed area	Marks
Task 1	Select Test Case Design Strategies	5
Task 2	Evaluate Random Testing	2
Task 3	Design and Evaluate Equivalence Classes	6
Task 4	Design and Evaluate Boundary Value Tests	4
Task 5	Apply Other Black-Box Techniques	3
Task 6	Design and Evaluate State-Transition Tests	3
Task 7	Construct the Control Flow Graph	4
Task 8	Evaluate Path Testing and Cyclomatic Complexity	4
Task 9	Judge Test Adequacy and Coverage	3
Task 10	Compare Black-Box and White-Box Effectiveness	4
Task 11	Recommend and Defend the Final Test Suite	4
Task 12	Day-4 Design Challenge (A–D, 2 marks each)	8
Total		**50**